



Trabalho de Compiladores

Ciência da Computação
Prof. Rodrigo Freitas Silva

Um Analisador Léxico para um dialeto da linguagem Pascal

1) Implemente um analisador léxico para a linguagem Pascal cuja gramática é descrita no Anexo I.

Siga as instruções e observações abaixo:

- a) Dê um nome para esta linguagem de programação
- b) Você deverá criar um IDE para o desenvolvimento e compilação dessa linguagem
- c) O programa não deverá diferenciar letras maiúsculas de letras minúsculas
- d) Os identificadores poderão ter até 15 caracteres e o primeiro caractere deverá ser obrigatoriamente uma letra. Os demais podem ser letras, números ou o underline (`_`)
 - i. Não é permitido caracteres especiais (`$%@#!? ...`), exceto o underline (`_`)
- e) As palavras chave são reservadas pela linguagem
- f) Os comentários deverão ser removidos do programa. Os comentários de múltiplas linhas nesta linguagem são escritos entre chaves ou entre parênteses e asterisco. Como no exemplo:


```
// Este é um comentário de uma linha

{ Aqui é
um comentário
de múltiplas linhas }
```

```
(* Aqui também
é um comentário
de múltiplas linhas *)
```
- g) Os espaços em branco também são automaticamente removidos pelo compilador
- h) Quando for encontrado um erro léxico, seu analisador deverá indicar a linha e a coluna do programa no qual o erro foi encontrado, assim como sublinhar de vermelho o erro no programa fonte
 - i. Você deve TENTAR encontrar esses erros léxicos não somente após o usuário mandar compilar o programa, mas também enquanto ele estiver implementando
- i) Lembre-se de incluir todos os tokens conhecidos antecipadamente na tabela de símbolos (palavras reservadas, sinais de pontuação, operadores). Inclua também os tokens encontrados no programa objeto (identificadores)
- j) Você deverá escolher pelo menos uma estratégia de recuperação de erros
- k) Deverá ser gerado o autômato finito determinístico para o fluxo de entrada dessa linguagem

- l) Ao final (na data marcada com o professor) , **você deverá apresentar seu trabalho e entregar um relatório** descrevendo o funcionamento do seu IDE e do seu analisador léxico, como foi o desenvolvimento do seu trabalho, o diagrama relacionando os módulos do seu programa, uma tabela de (tokens, lexemas, expressões regulares, atributos e valor) para cada token encontrado na linguagem.

2) Observações:

- As estruturas de controle **for** deverá ser incorporada a gramática por você na análise sintática
- Pode haver instruções com somente um **ID** ;
 - Avisar com '*Warning*' que a instrução não tem efeito nenhum na instrução durante a Análise Sintática
- **Ponto extra:** incluir novas funcionalidades como: tipo registro, tipo enumeração, etc.

ANEXO I

programa → **PROGRAM** *ID*; *declaracoes* **BEGIN** *instrucoes* **END** .

bloco → **BEGIN** *instrucoes* **END** ;

declaracoes → *declaracaoVariavel* *declaracaoConstante* *declProcedimento*

declaracaoConstante → **CONST** *declConsList* | €

declConsList → **ID** : *tipo* = *valor*; *declConsList* | **ID** = *valor*; *declConsList* | €

declaracaoVariavel → **VAR** *declVarList* | €

declVarList → *declVar* *declVarList* | €

declVar → *variavel* *conjuntoIds* : *tipo* ;

conjuntoIds → , *variável* *conjuntoIds* | €

tipo → **INTEGER** | **REAL** | **CHAR** | **STRING**

valor → *unario* | **LITERAL**

declProcedimento → *declProc* *declProcedimento* | €

declProc → **PROCEDURE** *ID*(*parametros*); *declaracaoVariavel* *bloco* |

FUNCTION *ID*(*parametros*) : *tipo*; *declaracaoVariavel* *bloco*

parametros → *declVarList* | €

instrucoes → *inst* *instrucoes* | €

inst → **ID** := *expr* ; | **ID** [*expr*] := *expr* ; | **ID** (*parametros2*) ; |

IF *expr* **THEN** *inst* | **IF** *expr* **THEN** *inst* **ELSE** *inst* |

WHILE *expr* **DO** *inst* | **REPEAT** *inst* **UNTIL** *expr* ; |

BREAK ; | **CONTINUE** ; | *bloco*

parametros2 → *expr* *parametros2* | , *expr* *parametros2* | €

expr → *exprComparacao* *expr2*

expr2 → **OU** *exprComparacao* *expr2* | **E** *exprComparacao* *expr2* | €

exprComparacao → *exprOp* *exprComparacao2*

exprComparacao2 → = *exprOp* *exprComparacao2* | <> *exprOp* *exprComparacao2* |

< *exprOp* *exprComparacao2* | <= *exprOp* *exprComparacao2* |

> *exprOp* *exprComparacao2* | >= *exprOp* *exprComparacao2* | €

exprOp → *termo* *exprOp2*

exprOp2 → + *termo* *exprOp2* | - *termo* *exprOp2* | €

termo → *unario* *termo2*

termo2 → * *unário* *termo2* | / *unário* *termo2* | €

unario → + *fator* | - *fator* | *fator*

fator → (*expr*) | *variavel* | **NUM** | **LITERAL**

variavel → **ID** | **ID**[*exprOp*]

NUM → *digitos* | *digitos*.*digitos*

digitos → *dig* *digitos**

dig → [0-9]

ID → [A-Za-z][*letra* | *dig* | _]*

LITERAL → '[*letra* | *dig* | *CARACTER_ESPECIAL*]*'